

TP7 : Shaders

Hetic – 2026

L'objectif de ce TP est de vous familiariser sur quelques aspects des shaders. Il sera composé de petits exercices guidé ou vous devrez utiliser un shader pour réaliser différents effets

Exercice 1 : Shader toy

<https://www.shadertoy.com/view/3tKBDz>

Prenez ce shader et appliquez le sur un plane pour simuler un océan en utilisant un shaderMaterial

Remarque :

Shadertoy n'utilise que le fragment shader, vous devez quand meme déclarer un vertexShader très basique

Les code de shadertoy utilisent une variable iTime pour animer leur shader. Passer iTime en paramètre du shader grace à un uniform

```
const material = new THREE.ShaderMaterial({
  uniforms: {
    iTime: { value: 0.0 },
    iResolution: { value: new THREE.Vector3(window.innerWidth, window.innerHeight, 1.0) },
  },
  vertexShader: `
    varying vec2 vUv;

    void main() {
      vUv = uv;
      gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0);
    }
  `,
  fragmentShader: `
    uniform float iTime;
    uniform vec3 iResolution;
    uniform vec4 iMouse;
    varying vec2 vUv;

    void main() {
      .. code shader toy
    }
  `
});
```

Exercice 2 : Shader Simples

Ecrire un shader qui prend une texture en uniform et qui la met en noir et blanc

Ecrire un shader similaire qui peut éclairer ou assombrir une texture

Ecrire un shader qui fait clignoter une couleur

Exercice 3 : Amélioration du système de particules

Le material THREE.PointMaterial est très simple et ne permet que de modifier la position des particules. En utilisant un shaderMaterial, on peut créer un material qui permet de modifier la couleur, l'opacité, et l'angle de rotation

voici le VertexShader

```
this.vertexShader = `
    uniform float pointMultiplier;

    attribute float size;
    attribute vec3 colour;
    attribute float opacity;
    attribute float angle;

    varying vec3 vColour;
    varying float o;
    varying vec2 vAngle;

    void main() {
        vec4 mvPosition = modelViewMatrix * vec4(position, 1.0);

        gl_Position = projectionMatrix * mvPosition;
        gl_PointSize = size * pointMultiplier / gl_Position.w;

        vAngle = vec2(cos(angle), sin(angle));
        vColour = colour;
        o = opacity;
    }
`;
```

et le fragment shader

```
this.fragmentShader = `
    uniform sampler2D diffuseTexture;

    varying vec3 vColour;
```

```

    varying vec2 vAngle;
    varying float o;

    void main() {
        vec2 coords = (gl_PointCoord - 0.5) * mat2(vAngle.x, vAngle.y, -vAngle.y, vAngle.x)
+ 0.5;
        gl_FragColor = vec4(vColour, texture2D(diffuseTexture, coords).w * o) ;
    }
};

```

A partir de ce code, créer une animation d'explosion qui dure 3 secondes

<https://jsfiddle.net/60ht59ue/>

Exercice 4 : Vertex Shader

On souhaite maintenant créer un shader qui va simuler une herbe qui bouge avec le vent

<https://jsfiddle.net/>

```

const geometry = new THREE.PlaneGeometry(0.2, 1.0, 1, 12);

geometry.translate(0, 0.5, 0);

const material = new THREE.ShaderMaterial({
    side: THREE.DoubleSide,
    uniforms: {
        time: { value: 0.0 }
    },
    vertexShader: `
        uniform float time;
        varying vec2 vUv;

        void main() {
            vUv = uv;

            vec3 transformed = position;

            float heightFactor = clamp(position.y / 1.0, 0.0, 1.0);

            transformed.x += sin(time * 2.0 + position.y * 0.5) * 0.12 * heightFactor;
            transformed.z += cos(time * 2.0 + position.y * 0.5) * 0.12 * heightFactor;

            gl_Position = projectionMatrix * modelViewMatrix * vec4(transformed, 1.0);
        }
    `,
    fragmentShader: `
        varying vec2 vUv;
    `
});

```

```

void main() {
    vec3 bottomColor = vec3(0.08, 0.35, 0.08);
    vec3 topColor = vec3(0.35, 0.85, 0.25);

    vec3 color = mix(bottomColor, topColor, vUv.y);
    gl_FragColor = vec4(color, 1.0);
}

};

const grass = new THREE.Mesh(geometry, material);
scene.add(grass);

```

1 - Utiliser ce code pour créer un brin d'herbe sur une scene, vous devez mettre à jour le temps dans la loop d'affichage

2 - Expliquer ce que fait ce FragmentShader

```

vec3 bottomColor = vec3(0.08, 0.35, 0.08);
vec3 topColor = vec3(0.35, 0.85, 0.25);

vec3 color = mix(bottomColor, topColor, vUv.y);
gl_FragColor = vec4(color, 1.0);

```

3 - sur la geometry du plane

```

const geometry = new THREE.PlaneGeometry(0.2, 1.0, 1, 12);

```

A quoi correspond le 4 paramètre? Modifier ce paramètre par un chiffre plus bas. Comment cela influence t'il le shader?

Exercice 5: Combiner des textures

Ecrire un ShaderMaterial qui prend en uniform 2 textures. Votre shader doit combiner la couleur des 2 textures.

Utilisez les fonctions **mix** et **texture2d** de GLSL

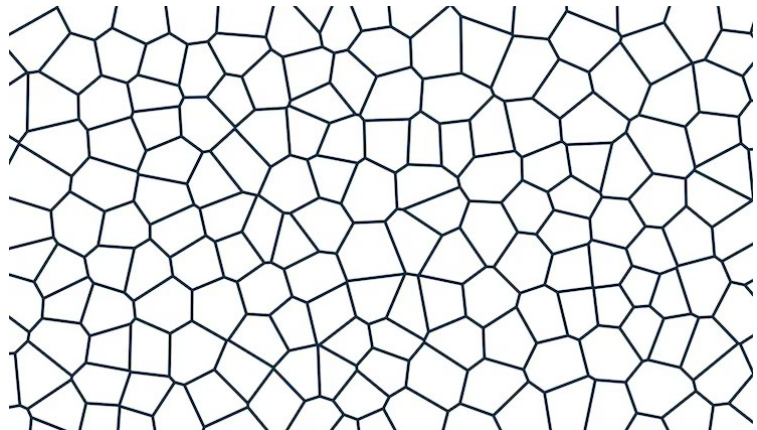
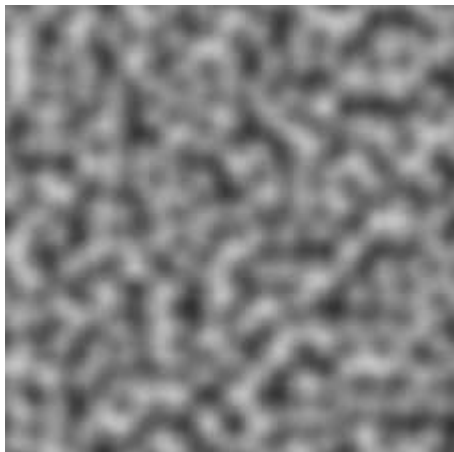


Exercice 6: Mask et Noise

Utiliser des couleurs ou des textures trop lisses rendent les scènes moins réalistes. Il est courant de devoir rajouter des imperfections aux couleurs pour les rendre plus organiques. Une des techniques est d'utiliser des variations aléatoires que l'on appelle des « Noise »

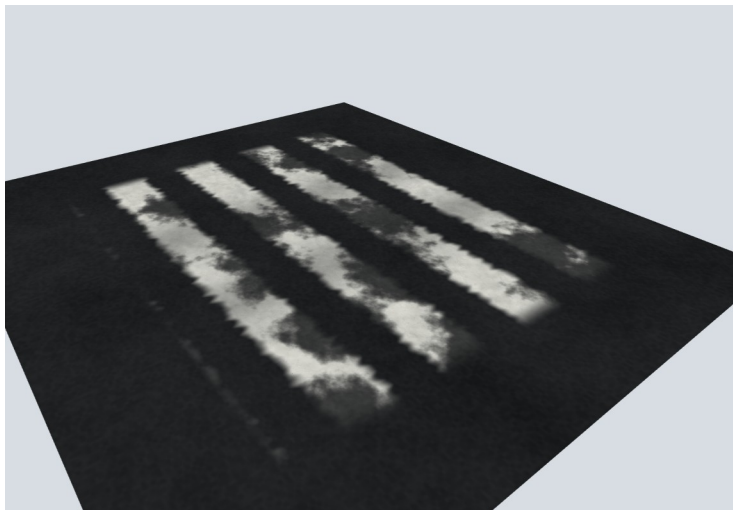
Le bruit de Perlin ou le bruit de Voronoï, sont des fonctions mathématiques qui génèrent des variations pseudo-aléatoires continues dans l'espace. En shader, ils permettent de casser l'aspect trop uniforme d'une couleur, d'une texture ou d'une surface, en ajoutant des irrégularités qui évoquent davantage des matériaux naturels comme la pierre, l'eau, les nuages, la fumée ou la terre.

Perlin Noise et Diagramme de Voronoï

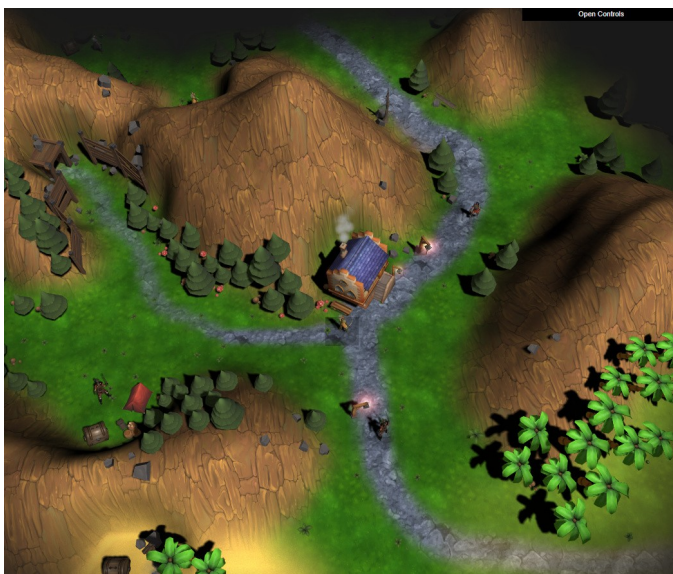


Le bruit de Perlin produit des variations douces et progressives, utiles pour simuler des dégradés organiques ou des reliefs subtils, tandis que le bruit de Voronoï génère des motifs cellulaires ou fragmentés, intéressants pour évoquer des craquelures, des alvéoles ou des structures minérales.

En utilisant un bruit de perlin , simuler une texture un peu effacé en utilisant un shaderMaterial et une texture de base. Votre shader doit utiliser le bruit de perlin pour modifier l'alpha de la couleur finale.



Bonus : Mask et Texture Splatting



https://www.youtube.com/watch?v=_wwcdVjvCpQ

Ecrire un shader qui prend en uniform 2 textures et un mask: Le shader doit combiner les 2 textures en fonction du mask